Training Leaves Traces: Weight-Level Model Provenance Without Watermarks

Anonymous submission

Abstract

Model provenance depends on external evidence—hashes, metadata, logs, or watermarks—which can fail when checkpoints are adapted or records are incomplete. We ask whether provenance can be inferred from weights alone: if a suspect checkpoint descends from a reference model, does training leave a persistent structural signal? We find that modern residual architectures exhibit such a signal. In a two-layer residual branch with input projection W_{in} and output projection W_{out}, training couples paired matrices so that $W_{\text{out}}W_{\text{in}}$ exhibits a diagonal-dominance fingerprint: a signed, identity-like component absent from mismatched or random pairings. For deeper branches, we measure the trace on $M = W_K \cdots W_1$, formalize it with $s(M) = |\text{tr}(M)|/\|M\|_F$, derive a separation margin, and compute cross-block score matrices that recover residual-branch correspondences and yield checkpoint-level ancestry evidence.

Empirically, across X architectures, Y model families, Z post-training weight modifications, and N reference–suspect pairs, the fingerprint recovers X–Y% residual-block correspondences and achieves X AUROC / Y% TPR at Z% FPR for descendant detection. It is absent at initialization, appears after training, strengthens with width, and persists under fine-tuning, quantization, pruning, weight noise, and LoRA-style adaptation. When perturbations substantially degrade the fingerprint, model utility also drops by X%. Boundary cases clarify the signal: independently trained models do not falsely match, while distillation removes the trace despite functional imitation.

1 Introduction

The open-weight ecosystem has grown explosively: Hugging Face now hosts nearly one million model repositories, with derivatives proliferating faster than originals (Hugging Face 2024). When Meta's LLaMA weights leaked in March 2023, the community produced instruction-tuned variants—Alpaca, Vicuna, Koala—within weeks, each costing under $300 to create (Touvron et al. 2023). This rapid, low-cost adaptation means any widely-distributed checkpoint spawns hundreds of fine-tuned, merged, quantized, and renamed descendants whose lineage quickly becomes untraceable.

Current approaches to model provenance rely on externally maintained evidence. Model cards document training datasets and licensing terms (Mitchell et al. 2019); experiment tracking platforms such as MLflow and Weights & Biases record artifacts and lineage graphs; cryptographic hashes provide integrity checks for unmodified files. For cases where theft is anticipated, watermarking methods embed ownership signals into network parameters (Uchida et al. 2017) or decision boundaries (Adi et al. 2018) that can later be extracted to verify provenance.

Yet each approach has fundamental limitations. Cryptographic hashes become invalid after any weight modification—including routine quantization or fine-tuning. Metadata and provenance logs can be falsified, lost, or simply never recorded. Watermarks require deliberate insertion before deployment and remain vulnerable to removal: a recent systematization found that no surveyed DNN watermarking scheme proved robust against fine-tuning, pruning, or model extraction attacks (Lukas et al. 2022). These limitations leave a critical gap: there is no established method to verify provenance for models released without embedded watermarks or whose external records are unavailable.

Prior intrinsic fingerprinting methods have asked whether weight statistics—means, variances, or histogram features—can identify models (Zheng et al. 2022; Zhao et al. 2020), but these aggregate properties lack verification precision and are easily disrupted. Training dynamics research has established that gradient descent enforces near-orthogonal Jacobians in residual networks (Pennington, Schoenholz, and Ganguli 2017; Saxe, McClelland, and Ganguli 2014), yet this work focused on trainability, not on what structural traces the process leaves behind. We ask a different question: *does training itself imprint a recoverable signature—not in aggregate statistics, but in the relational structure between paired weight matrices?*

The answer is yes. In residual architectures, trained residual-branch products exhibit diagonal-dominant structure. This phenomenon emerges because residual blocks must maintain near-identity behavior for stable gradient flow—a requirement formalized as *dynamical isometry*, where the input-output Jacobian's singular values concentrate near unity throughout training (Pennington, Schoenholz, and Ganguli 2017; Saxe, McClelland, and Ganguli 2014). For a residual block computing $x + W_{\text{out}}\phi(W_{\text{in}}x)$, this constraint forces the branch product toward a negative scaled identity: $W_{\text{out}}W_{\text{in}} \approx -\varepsilon I$ (He et al. 2016). We find this fingerprint is absent at initialization, emerges

within the first five epochs of training, and yields a diagonal-dominance score that scales as $\Theta(\sqrt{d})$—separating correctly paired blocks from mismatched pairings by a margin that grows linearly with hidden dimension d.

This trace enables *passive* weight-level provenance verification: auditing whether a suspect checkpoint descends from a reference model without requiring watermarks, metadata, or cooperation from the original trainer. Unlike embedded watermarks that must be inserted before or during training, the diagonal-dominance fingerprint emerges from optimization itself and can be read retroactively from any residual model—including checkpoints deployed long before provenance verification was anticipated. The fingerprint maintains 100% pair accuracy across 21 attack configurations (fine-tuning, quantization, pruning, weight noise up to $\sim$20%), degrading only when perturbations also destroy model utility. Boundary cases clarify the signal's scope: independently trained models do not falsely match, while distillation removes the trace—correctly, since the method detects weight-level lineage rather than functional imitation.

We make the following contributions:

- **We formalize the diagonal-dominance signal and derive a separation margin that grows with width.** Building on Park (2026)'s observation that training induces negative diagonal structure in residual-branch products, we define the score $s(i,j) = |\mathrm{tr}(M)|/\|M\|_F$ and prove it achieves $\Theta(\sqrt{d})$ for correct pairs versus $\Theta(1/\sqrt{d})$ for mismatched pairs (Section 3).
- **We develop a passive provenance verification protocol.** Cross-block score matrices combined with Hungarian matching recover residual-branch correspondences and yield checkpoint-level ancestry evidence from weights alone—without watermarks, metadata, or cooperation from the original trainer (Section 4).
- **The fingerprint generalizes across architectures and survives post-training modification.** We validate across nine architecture families (GPT-2 to 1.5B, BERT, LLaMA, Mistral, Qwen, Gemma, ViT, Whisper, ResNet), demonstrate robustness under 21 attack configurations, and confirm via ablation that the signal requires both training and residual connections (Sections 5–6).

2 Background and Problem Setting

2.1 Passive Model Provenance

We define *passive model provenance* as follows: given a reference checkpoint R and a suspect checkpoint S with missing or untrusted metadata, determine whether S shares weight-level lineage with R after allowed post-training transformations. This is a *white-box* verification task—the verifier has access to both sets of weights. The setting matches regulatory audits, model hub compliance checks, and ownership disputes where both parties can produce their checkpoints.

The method detects *weight-level lineage*, not behavioral equivalence. Two models share weight-level lineage if the suspect's parameters can be derived from the reference through transformations such as fine-tuning, pruning, or quantization—without retraining from scratch. This is distinct from functional equivalence, where models produce similar outputs regardless of whether their weights share any relationship. Model extraction attacks (Tramèr et al. 2016) create functional copies with entirely fresh weights; such models are behaviorally similar but share no weight-level identity.

Distilled models and independently trained functional copies are explicitly outside the positive guarantee. A student trained via knowledge distillation (Hinton, Vinyals, and Dean 2015) may replicate the teacher's behavior with high fidelity, but it evolves its weights through a separate optimization process that produces distinct structural fingerprints. This boundary is intentional: proving that weights were copied or derived requires distinguishing "same functionality" from "same parameters (up to transformation)."

2.2 Limits of Existing Methods

Model provenance verification has relied on three families of approaches, each with fundamental limitations.

Cryptographic hashes fail immediately upon any weight modification. The avalanche effect ensures that fine-tuning, pruning, or even numerical precision changes produce entirely different digests, providing no signal of derivation.

Metadata and logging systems—model cards (Mitchell et al. 2019), experiment trackers like MLflow and Weights & Biases—require trusted provenance records that may be incomplete, falsified, or absent when models are redistributed outside controlled environments.

Watermarking methods embed ownership signals into weights (Uchida et al. 2017; Rouhani, Chen, and Koushanfar 2019) or train models to produce specific outputs on trigger inputs (Adi et al. 2018; Zhang et al. 2018). Both require forethought: the watermark must be inserted before training concludes, precluding retroactive verification of already-deployed models. A systematic evaluation found that no surveyed watermarking scheme is robust in practice against fine-tuning, pruning, and model extraction attacks (Lukas et al. 2022).

Output watermarking systems such as SynthID (Dathathri et al. 2024) identify AI-generated content by embedding statistical signatures during generation, but address a fundamentally different question: they detect whether content was AI-generated, not whether one model's weights derive from another's.

Decision-boundary fingerprinting (Cao, Jia, and Gong 2021; Le Merrer, Perez, and Trédan 2020) identifies functional similarity via adversarial examples near classification boundaries, but cannot distinguish weight lineage: a distilled student may share decision surfaces while having entirely independent weights.

The key differentiator of our approach: prior watermarking work asks how to *insert or protect* an ownership signal. We ask whether training already *leaves* a signal that can be read retroactively.

2.3 Residual Branch Products

Modern deep networks use *residual connections*—shortcuts that allow information to bypass layers—to enable stable

Architecture	Residual Form	Product M
BasicBlock / MLP	$x + W_2\phi(W_1 x)$	$W_2 W_1$
Transformer MLP	$x + W_2\phi(W_1 x)$	$W_2 W_1$
Attention V/O	$x + W_O \mathrm{Attn}(W_V x)$	$W_O W_V$
Attention Q/K	$\mathrm{softmax}(QK^\top)$	$W_Q W_K^\top$
Bottleneck	$x + W_3\phi(W_2\phi(W_1 x))$	$W_3 W_2 W_1$
SwiGLU MLP	gated linear unit	$W_\mathrm{d} W_\mathrm{u}$

Table 1: Architecture-aware factorization for residual branch products.

training of very deep architectures (He et al. 2016). A residual block computes $x' = x + F(x)$, where x is the input and F is the residual branch function. For these blocks to maintain stable gradient flow, the Jacobian $J = I + J_F$ must remain close to orthogonal throughout training—a condition formalized as *dynamical isometry* (Pennington, Schoenholz, and Ganguli 2017; Saxe, McClelland, and Ganguli 2014).

For a residual branch with K linear transformations separated by nonlinearities, we define the *residual branch product*:

$$M = W_K W_{K-1} \cdots W_1 \in \mathbb{R}^{d\times d} \tag{1}$$

This product captures how the branch transforms the input space. The dynamical isometry constraint implies that M should approximate a negative scaled identity: $M \approx -\varepsilon I$ for small $\varepsilon > 0$ (Tarnowski et al. 2019; De and Smith 2020).

Table 1 shows the architecture-aware factorization for common residual blocks. Using the correct factorization is critical: the naive endpoint product $W_3 W_1$ for Bottleneck ResNets (skipping the middle convolution) yields chance-level pairing accuracy, while the full triple product $W_3 W_2 W_1$ recovers 91–100% of correct pairs.

3 Training-Induced Diagonal Dominance

3.1 Empirical Observation

Training a deep residual network imprints a characteristic structure on its weights. We first demonstrate this phenomenon empirically before formalizing it.

At initialization, no diagonal structure appears. We compute the score matrix $s(i,j)$ for all pairs of input and output projections in a 48-block residual network. At epoch 0, the matrix shows no discernible pattern: Hungarian matching recovers only 6.25% of correct pairs (chance is $1/48 \approx 2.1\%$), and pair separation—the gap between the worst correct-pair score and the best incorrect-pair score—is -0.42 (negative indicates no separation). Across seven initialization schemes—Kaiming uniform/normal, Xavier uniform/normal, Gaussian, uniform, and orthogonal—pair accuracy remains at chance level ($\leq 2.1\%$). Critically, orthogonal initialization provides dynamical isometry at initialization by construction (Saxe, McClelland, and Ganguli 2014), yet shows *zero* correct pairs before training, ruling out the hypothesis that near-orthogonal Jacobians alone create the fingerprint.

During training, diagonal structure emerges rapidly. By epoch 5, the diagonal of $s(i,j)$ is fully separated and Hungarian matching recovers 100% of correct pairs. The mean correct-pair score rises monotonically from 0.19 at epoch 0 to 2.07 at epoch 300, while the mean incorrect-pair score remains flat at $\approx$0.16—a 21$\times$ ratio at convergence. The transformation occurs within the first few epochs, well before training loss plateaus.

The effect is not explained by matrix shape or architecture alone. A null model with randomly initialized weights achieves only 6.25% pair accuracy with negative separation, ruling out shape compatibility as an explanation. To confirm the fingerprint requires residual training specifically, we train a PlainNet—architecturally identical except that skip connections are removed. While both networks converge to similar evaluation loss (1.35 vs 1.56), ResNet achieves 100% pair accuracy (AUC 0.98) with 100% of correctly paired products exhibiting negative trace, while PlainNet achieves only 3%—at the chance baseline of $1/24 \approx 4.2\%$—with AUC 0.51 and no discernible diagonal structure. The skip connection creates a functional constraint—the block must act as a near-identity perturbation to preserve gradient flow—that imprints the characteristic trace structure.

3.2 Diagonal-Dominance Fingerprint

For a candidate residual branch product $M_{ij} = W_{\mathrm{out}}^{(j)} W_{\mathrm{in}}^{(i)}$, we define the *diagonal-dominance score*:

$$s(i,j) = \frac{|\mathrm{tr}(M_{ij})|}{\|M_{ij}\|_F} \tag{2}$$

The numerator $|\mathrm{tr}(M)|$ captures *diagonal mass*—how much of the matrix energy concentrates on the diagonal. The denominator $\|M\|_F$ normalizes by total matrix energy (Frobenius norm), isolating structural fingerprint from scale.

This ratio has a natural geometric interpretation. A matrix with energy uniformly spread across entries achieves $s = \Theta(1/\sqrt{d})$, while a matrix proportional to the identity achieves $s = \sqrt{d}$. The key insight is that correctly paired blocks—where all K matrices come from the same residual branch—score at $\Theta(\sqrt{d})$, while incorrect pairings remain at the random baseline $\Theta(1/\sqrt{d})$. This yields a signal-to-noise gap that grows linearly with hidden dimension d, making correct pairs increasingly separable in wider networks.

Dynamical isometry (Pennington, Schoenholz, and Ganguli 2017) predicts that a well-trained residual block has Jacobian $J = I + W_{\mathrm{out}} W_{\mathrm{in}}$ close to orthogonal. For the block to act as a near-identity perturbation of the residual stream, the branch product must approximate a negative scaled identity. We formalize this with the decomposition:

$$M = W_{\mathrm{out}}^{(i)} W_{\mathrm{in}}^{(i)} = -\varepsilon I + E \tag{3}$$

where $\varepsilon = |\mathrm{tr}(M)|/d > 0$ captures diagonal strength and E is the residual with $\mathrm{tr}(E) = 0$. The matrix E contains all off-diagonal structure plus the zero-trace portion of the diagonal. Dynamical isometry implies $\|E\|_F \ll \varepsilon\sqrt{d}$ for well-trained blocks; the following proposition quantifies the margin exactly.

Crucially, diagonal dominance captures *relational* structure between paired matrices—how they jointly encode the

near-identity mapping—rather than aggregate properties of individual matrices. This relational fingerprint is preserved under fine-tuning and noise because it reflects the functional constraint that the residual block must approximate.

3.3 Margin Analysis and Null Model

Proposition 1 (Margin Formula)**.** *Let $M = -\varepsilon I + E$ with $\varepsilon = |\mathrm{tr}(M)|/d$ and $\mathrm{tr}(E) = 0$. Then the diagonal-dominance score satisfies*

$$s(i,i) = \frac{\sqrt{d}}{\sqrt{1 + \|E\|_F^2/(\varepsilon^2 d)}} \leq \sqrt{d}, \tag{4}$$

with equality if and only if $E = 0$, i.e., $M = -\varepsilon I$.

Proof. By construction, $|\mathrm{tr}(M)| = \varepsilon d$. For the Frobenius norm, expand $\|M\|_F^2 = \|-\varepsilon I + E\|_F^2 = \varepsilon^2 d - 2\varepsilon\,\mathrm{tr}(E) + \|E\|_F^2 = \varepsilon^2 d + \|E\|_F^2$, where the last step uses $\mathrm{tr}(E) = 0$. Substituting into (2): $s(i,i) = \varepsilon d/\sqrt{\varepsilon^2 d + \|E\|_F^2} = \sqrt{d}/\sqrt{1 + \|E\|_F^2/(\varepsilon^2 d)}$. The bound follows immediately, with equality iff $\|E\|_F = 0$. □

The margin formula reveals the geometry of diagonal dominance. The numerator $\sqrt{d}$ is the score achievable by a perfect negative-scalar identity; the denominator penalizes deviation from this ideal via the dimensionless ratio $\|E\|_F^2/(\varepsilon^2 d)$. For typical trained blocks, $\|E\|_F/(\varepsilon\sqrt{d}) \in [1.9, 3.8]$, yielding $s(i,i) \in [1.76, 3.23]$—well above the baseline but below the theoretical maximum of $\sqrt{d}$.

Corollary 1 (Signal-to-Baseline Ratio)**.** *Suppose $(W_{\text{in}}^{(i)}, W_{\text{out}}^{(j)})$ with $i \neq j$ are independent random matrices with i.i.d. zero-mean entries. Then $\mathbb{E}[s(i,j)^2] = 1/d$, so $\mathbb{E}[s(i,j)] \approx 1/\sqrt{d}$. The signal-to-baseline ratio scales as d.*

Correct pairs achieve scores of order $\sqrt{d}$, while incorrect pairs score at the random baseline $1/\sqrt{d}$. The gap grows with dimension, providing increasingly robust separation for wider networks. On GPT-2 models, mean $s(i,i)$ rises from 4.18 ($d = 768$, 124M parameters) to 7.77 ($d = 1600$, 1.5B parameters), consistent with $\sqrt{d}$ scaling.

Corollary 2 (Null Model)**.** *For randomly initialized weights with no training, all pairs satisfy $\mathbb{E}[s(i,j)] = 1/\sqrt{d} + o(1)$ uniformly in (i,j). Hungarian matching recovers correct pairs at the chance rate $1/L$.*

The null model rules out three alternative explanations: (a) diagonal dominance is not an artifact of compatible matrix shapes; (b) the residual connection itself does not create the signal; (c) any nontrivial loss level does not suffice. The diagonal-dominance fingerprint is a *learned* property induced by gradient descent. The theory explains why the observed structure is identifiable once induced; controlled experiments—including the initialization ablation showing orthogonal init at 0% pairing and the PlainNet control at 3%—confirm that training induces it

4 From Fingerprints to Provenance Verification

4.1 Pairing Algorithm

Purpose: Turn the fingerprint into a concrete computational method.

Content: Algorithm:

1. Extract candidate residual-branch matrices.
2. Compute all pairwise scores $s(i,j)$.
3. Construct a score matrix.
4. Use Hungarian matching or equivalent assignment.
5. Recover block correspondences.

4.2 Model-Level Lineage Score

Purpose: Bridge the biggest current gap: pair accuracy is not enough for AAAI Main.

Content: Define a model-level lineage score. Possible forms:

$$L(A,B) = \frac{1}{|P|} \sum_{p \in P} \|s_p(A,B)\| \tag{5}$$

or:

$$L(A,B) = \frac{\mu_{\text{matched}}(A,B) - \mu_{\text{null}}}{\sigma_{\text{null}}} \tag{6}$$

The exact formula can change, but the paper needs a scalar decision object.

Report against:

- Known descendants
- Unrelated independently trained models
- Random-initialized models
- Functionally similar but non-descendant models
- Distilled models

4.3 Verification Protocol

Purpose: Make the paper feel like a complete verification framework, similar in maturity to accepted watermarking papers.

Content: Protocol:

1. **Reference:** start from a known checkpoint.
2. **Suspect:** receive a candidate checkpoint with untrusted metadata.
3. **Extract:** compute architecture-aware residual branch products.
4. **Align:** compute diagonal-dominance alignment.
5. **Aggregate:** produce model-level lineage score.
6. **Calibrate:** compare to unrelated-model null.
7. **Decide:** output descendant / non-descendant / inconclusive.

5 Experiments

5.1 Does Training Induce the Fingerprint?

Purpose: Establish existence and rule out trivial explanations.

Experiments:

- Untrained vs trained networks
- Training trajectory
- Initialization ablations
- Random seed ablations

- Orthogonal initialization control
- Non-residual or weak-residual controls where feasible

Metrics:
- Pair accuracy
- Correct-vs-incorrect score separation
- AUC
- Mean correct score over training
- Mean incorrect score over training

5.2 Does It Generalize Across Architectures?

Purpose: Show this is not a toy-network artifact.

Experiments: Evaluate across:
- Residual MLPs
- GPT-style transformer MLPs
- Transformer attention V/O and Q/K paths
- BERT-style encoders
- LLaMA/Mistral/Qwen/Gemma-style architectures if available
- ViT
- ConvNeXt
- ResNet BasicBlock and Bottleneck
- Optionally Whisper/audio if already feasible

Metrics:
- Pair accuracy
- AUC
- Pair separation
- Negative trace fraction
- Architecture-specific failure cases

5.3 Does It Scale With Model Size?

Purpose: Turn the work into empirical ML theory, not just forensics.

Experiments: Use model families with multiple sizes, such as GPT-2 small/medium/large/XL or comparable open models.

Report:
- Hidden dimension
- Number of layers
- Mean $s(i, i)$
- Pair accuracy
- Score separation
- Scaling trend versus $\sqrt{d}$

5.4 Does It Survive Post-Training Modification?

Purpose: Show the fingerprint is useful for passive provenance.

Experiments: Evaluate descendants after:
- Same-task fine-tuning
- Alternative-task fine-tuning
- LoRA/adapters merged into the base model
- Quantization
- Pruning
- Weight noise
- Partial layer replacement
- Model soups / weight averaging, if feasible

Report model utility alongside fingerprint survival.

Metrics:
- Lineage score
- Pair accuracy
- AUROC / TPR at fixed FPR
- Task accuracy/perplexity/utility
- Utility-vs-fingerprint degradation curve

5.5 How Does It Compare to Baselines?

Purpose: Prevent the obvious reviewer criticism that the baselines are too weak.

Baselines (minimum):
- Frobenius matching
- Singular-value distance
- Weight cosine similarity
- Normalized/centered weight similarity
- Spectral-statistic fingerprints
- Activation similarity such as CKA/SVCCA-style comparisons
- Behavioral agreement
- Existing watermark/fingerprint methods where applicable

Metrics:
- Pair accuracy
- AUROC for lineage detection
- FPR at high TPR
- Robustness under transformations
- Runtime

6 Adaptive Evasion and Boundary Cases

6.1 Direct Fingerprint-Suppression Attacks

Purpose: Reviewer-proof the paper against the obvious attack.

Content: Attack objective:

$$\min_{\Delta W} L(A, A + \Delta W) \tag{7}$$

subject to preserving model utility.

Evaluate whether an adversary can reduce the lineage score while keeping the model useful.

6.2 Function-Preserving Reparameterizations

Purpose: Address the basis-dependence objection.

Experiments / analysis: Consider:
- Orthogonal rotations
- Neuron/channel permutations
- Invertible basis changes where architecture permits
- Equivalent reparameterizations across adjacent layers
- Adapter insertion/removal

6.3 Distillation and Retraining

Purpose: Define the boundary cleanly.

Content: State:

> The fingerprint tracks weight-level lineage, not function-level imitation.

Evaluate:
- Distilled students
- Retrained-from-scratch models
- Independently trained models with same architecture
- Same architecture/same data/different seed

7 Related Work

Purpose: Situate the contribution without bloating the paper.

Recommended subsections:

1. Neural Network Watermarking

- Embedded parameter watermarks
- Backdoor/trigger watermarks
- Watermark capacity
- NeuralMark / robust weight-based watermarking

2. Model Fingerprinting and Ownership Verification

- Black-box fingerprints
- Decision-boundary fingerprints
- Intrinsic/statistical fingerprints

3. Training Dynamics and Residual Networks

- Dynamical isometry
- Residual near-identity behavior
- Weight-space structure

4. Model Provenance and Trustworthy ML

- Model supply chains
- Checkpoint auditability
- Lineage verification

8 Discussion and Limitations

Purpose: Show maturity and avoid overclaiming.

Must say clearly:

- White-box only
- Residual architectures only
- Detects weight-level lineage, not behavioral identity
- Distillation/retraining breaks or erases the signal
- Not a proof of safety
- Not a replacement for watermarking
- Adaptive reparameterization attacks require care
- Provenance is one part of an audit stack

9 Conclusion

Purpose: End with the scientific thesis.

Recommended message:

> Training itself can provide provenance signal. Residual-network optimization leaves an intrinsic weight-space trace; diagonal dominance exposes it; and this trace enables passive model-lineage auditing under common post-training transformations.

References

Adi, Y.; Baum, C.; Cisse, M.; Pinkas, B.; and Keshet, J. 2018. Turning your weakness into a strength: Watermarking deep neural networks by backdooring. In *27th USENIX Security Symposium*, 1615–1631.

Cao, X.; Jia, J.; and Gong, N. Z. 2021. IPGuard: Protecting Intellectual Property of Deep Neural Networks via Fingerprinting the Classification Boundary. In *AsiaCCS*, 14–25.

Dathathri, S.; See, A.; Ghaisas, S.; Huang, P.-S.; McAdam, R.; Welbl, J.; Bachani, V.; Kaskasoli, A.; Sherborne, T.; et al. 2024. Scalable Watermarking for Identifying Large Language Model Outputs. *Nature*.

De, S.; and Smith, S. L. 2020. Batch Normalization Biases Residual Blocks Towards the Identity Function in Deep Networks. In *NeurIPS*.

He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 770–778.

Hinton, G.; Vinyals, O.; and Dean, J. 2015. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*.

Hugging Face. 2024. Hugging Face Model Hub. https://huggingface.co/models. Accessed: 2024.

Le Merrer, E.; Perez, P.; and Trédan, G. 2020. Adversarial Frontier Stitching for Remote Neural Network Watermarking. *Neural Computing and Applications*, 32(13): 9233–9244.

Lukas, N.; Jiang, E.; Li, X.; and Kerschbaum, F. 2022. SoK: How robust is image classification deep neural network watermarking? In *2022 IEEE Symposium on Security and Privacy (SP)*, 787–804. IEEE.

Mitchell, M.; Wu, S.; Zaldivar, A.; Barnes, P.; Vasserman, L.; Hutchinson, B.; Spitzer, E.; Raji, I. D.; and Gebru, T. 2019. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 220–229.

Park, H. 2026. I Dropped a Neural Net. *arXiv preprint arXiv:2602.19845*.

Pennington, J.; Schoenholz, S.; and Ganguli, S. 2017. Resurrecting the sigmoid in deep learning through dynamical isometry: theory and practice. In *Advances in Neural Information Processing Systems*, volume 30.

Rouhani, B. D.; Chen, H.; and Koushanfar, F. 2019. DeepSigns: An End-to-End Watermarking Framework for Deep Neural Networks. In *ASPLOS*, 485–497.

Saxe, A. M.; McClelland, J. L.; and Ganguli, S. 2014. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. In *International Conference on Learning Representations*.

Tarnowski, W.; Warchol, P.; Jastrzebski, S.; Tabor, J.; and Nowak, M. 2019. Dynamical Isometry is Achieved in Residual Networks in a Universal Way for Any Activation Function. In *AISTATS*, 2221–2230.

Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.-A.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Azhar, F.; et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Tramèr, F.; Zhang, F.; Juels, A.; Reiter, M. K.; and Ristenpart, T. 2016. Stealing Machine Learning Models via Prediction APIs. In *USENIX Security Symposium*, 601–618.

Uchida, Y.; Nagai, Y.; Sakazawa, S.; and Satoh, S. 2017. Embedding watermarks into deep neural networks. In *Proceedings of the 2017 ACM on International Conference on Multimedia Retrieval*, 269–277.

Zhang, J.; Gu, Z.; Jang, J.; Wu, H.; Stoecklin, M. P.; Huang, H.; and Molloy, I. 2018. Protecting Intellectual Property of

Deep Neural Networks with Watermarking. In *ASIACCS*, 159–172.

Zhao, W.; Rao, S.; Li, J.; and Xiong, Z. 2020. Shaping deep feature space towards gaussian mixture for visual classification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 3255–3264.

Zheng, Z.; Zhang, H.; Chen, J.; and Li, B. 2022. Fingerprinting deep neural networks globally via universal adversarial perturbations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 13430–13439.